ChampionCHIP
eXperience

Guia de submissão

Desafio - Fase 2

Wernher von Braun
centro de pesquisas avançadas

ChipInventor
CLOUD BASED EDA PLATFORM

ChipInventor
Design
TEAM

Este documento define os entregáveis obrigatórios para a Fase 2 da ChampionCHIP eXperience.

Os participantes deverão apresentar a arquitetura, implementação, validação e resultados físicos do projeto desenvolvido, incluindo documentação técnica, resultados OpenLane e demonstração funcional do processador.

1. Visão Geral do Projeto

Descrever o objetivo geral do projeto, a arquitetura utilizada, os principais módulos implementados e o funcionamento geral do sistema. Explicar também quais funcionalidades foram implementadas no core RISC-V.

2. Instruções Suportadas

O participante deverá apresentar a cobertura das instruções implementadas no processador, agrupadas por categoria da ISA RISC-V utilizada no projeto.

Deverá ser incluída uma tabela resumo contendo a quantidade de instruções esperadas pela competição e a quantidade efetivamente implementada em cada categoria.

Tabela de Cobertura da ISA

Categoria	Instruções Esperadas	Instruções Implementadas	Cobertura (%)
Arithmetic and Logic (Register)	10	XX	XX%
Arithmetic and Logic (Immediate)	9	XX	XX%
Load	5	XX	XX%
Store	3	XX	XX%
Branch	6	XX	XX%
Jump	2	XX	XX%
Upper Immediate	2	XX	XX%
Sistema / Sincronização	3		
Multiplication	4	XX	XX%
CRC	3	XX	XX%

Total	**47**	**XX**	**XX%**

Além da tabela resumo, o participante deverá apresentar a lista das instruções implementadas em cada categoria.

Exemplo

Arithmetic and Logic (Register) (10/10)

ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND

Load (5/5)

LB, LH, LW, LBU, LHU

CRC (3/3)

CRCB, CRCH, CRCW

Por fim, deverá ser apresentada uma tabela contendo a codificação de todas as instruções implementadas, incluindo opcode, funct3 e funct7 (quando aplicável).

Exemplo de Codificação

Instrução	Opcode	Funct3	Funct7
ADD	0110011	000	0000000
SUB	0110011	000	0100000
MUL	0110011	000	0000001
CRCB	0110011	000	1000000

A tabela de cobertura será utilizada pelos avaliadores para verificar rapidamente o percentual de implementação da ISA suportada pelo processador.

3. Datapath

Apresentar o desenho completo do datapath do processador, junto à unidade de controle, mostrando o fluxo de dados entre os principais módulos do sistema.

4. Módulos e Testbenches

Explicar cada módulo desenvolvido bem como apresentar um diagrama de bloco realizado no ChipInventor para cada módulo e os testbenches individuais.

5. Resultados OpenLane

Apresentar os resultados obtidos de área total e densidade após o fluxo de implementação física utilizando OpenLane, bem como a imagem do GDSII final do projeto. Observação, na Fase 4 os participantes selecionados focarão apenas em aperfeiçoar o projeto.

6. Firmware Demo

Apresentar imagens do waveform que comprovem a execução correta do firmware de validação, bem como a saída de logs gerados pelo testbench, provando que a execução chegou ao final da validação sem erros.

Obs: O firmware oficial de validação será disponibilizado na plataforma ChampionCHIP. A nota será calculada até a parte que o firmware conseguir ser executado, ao executar todas as etapas a nota será 100% para essa categoria.

7. Video Demo

Produzir um vídeo demonstrando:

- Arquitetura do projeto
- Funcionamento do processador
- Simulação do sistema
- Execução do firmware
- Resultados do OpenLane
- Layout final do chip

O vídeo deve ter duração de 5 a 7 minutos disponível no YouTube.

8. Formato do Relatório

O relatório deve constar os tópicos de 1 a 6 desse documento no formato PDF, tendo com limite máximo de 20 páginas.

9. Arquivos Esperados

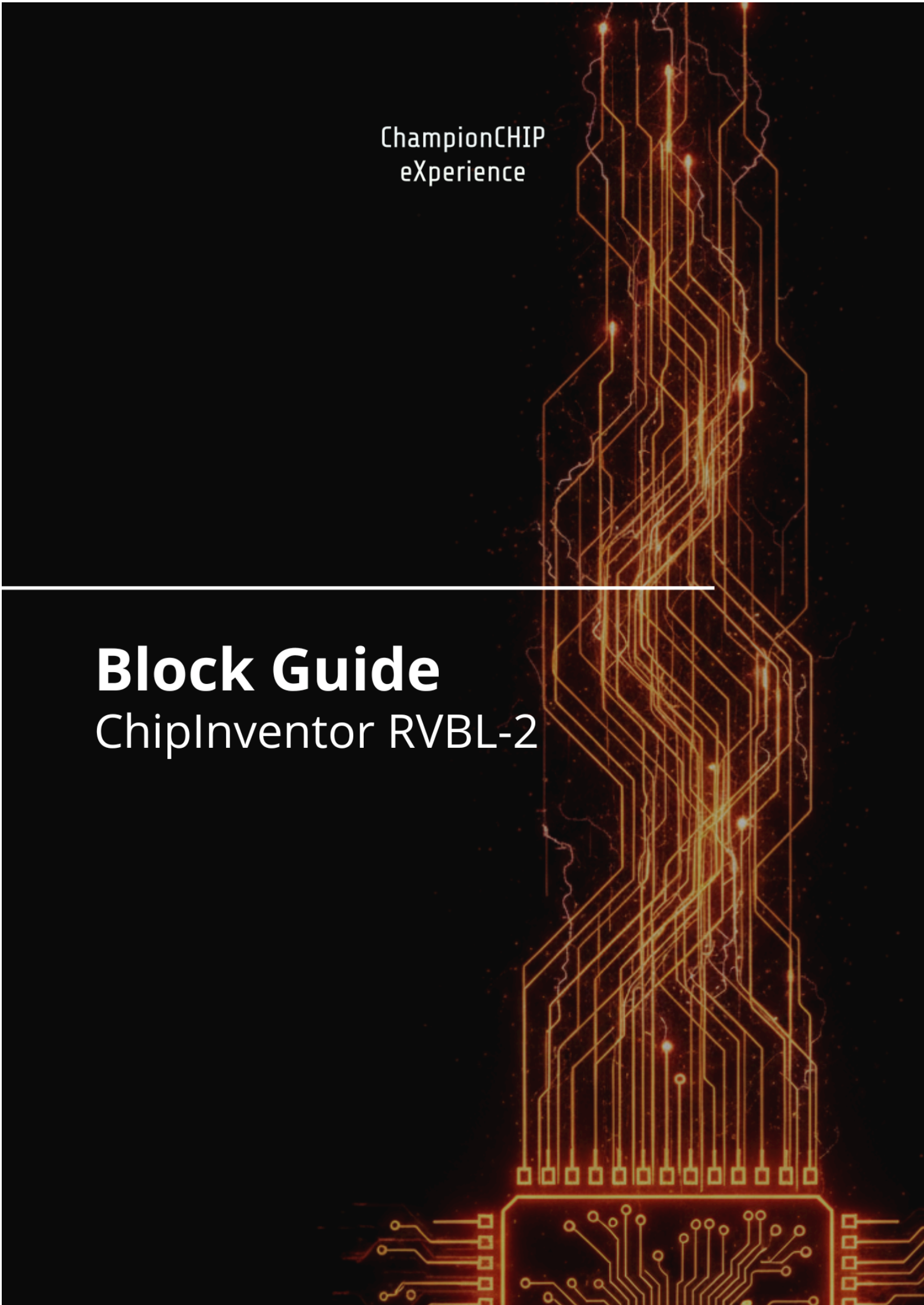
A lista de arquivos que devem ser enviados na submissão do projeto:

- Relatório
- Arquivo GDSII da Síntese OpenLane
- Arquivo GL (gate-level netlist) localizado na pasta results/final/verilog/gl
- Arquivo config.json da Síntese OpenLane
- Todo o código desenvolvido incluindo os arquivos de testbenches

ChampionCHIP
eXperience

Block Guide

ChipInventor RVBL-2



Índice

1. Contextualização.....3
1.1 Convenções 3
1.2 Referências 3
1.3 Acrônimos e Abreviações 3
1.4 Glossário 4
1.5 Notações 4
2. Introdução 7
2.1 O Núcleo 5
2.2 Descrição dos Pinos do IP 5
2.3 Estrutura Interna 6
2.4 Conjunto de Instruções 7
3. Núcleo RISC-V 11
3.1 *Datapath* do Núcleo 9
3.2 Unidade de Controle 12
3.3 LSU (Load/Store Unit) 12
4. Mapeamento de Memória 16
4.1 Mapa de Memória 16
4.2 Memória de Instruções (IMEM) 16
4.3 Memória de Dados (DMEM) 17
4.4 Decodificador de Endereços **18**

CONFIDENCIAL

1. Contextualização

Este documento apresenta as funcionalidades, detalhes e requisitos do IP de um microcontrolador de 32 bits baseado em RISC-V, com o objetivo de guiar sua utilização e replicação. Enquanto em desenvolvimento, as definições de suas funções servem como especificação para a implementação do IP.

1.1 Convenções

A Tabela 1 contém as convenções usadas neste documento.

Tabela 1. Lista de Convenções

Convenção	Definição
I	Sinal somente de entrada
O	Sinal somente de saída
I/O	Sinal de entrada ou saída
R	Acesso somente de leitura para bit ou byte de dado
W	Acesso somente de escrita para bit ou byte de dado
R/W	Acesso de leitura ou escrita para bit ou byte de dado

1.2 Referências

[1] The RISC-V Instruction Set Manual, Volume I; Unprivileged Architecture (2026).

1.3 Acrônimos e Abreviações

A Tabela 2 contém os acrônimos e abreviações usados neste documento.

Tabela 2. Acrônimos e Abreviações

Termo	Definição
MSB	*Most Significant Bit* – Bit Mais Significativo
LSB	*Least Significant Bit* – Bit Menos Significativo
ISA	*Instruction Set Architecture* - Arquitetura do Conjunto de Instruções
CRC	*Cyclic Redundant Check*
ULA	Unidade Lógica e Aritmética (ou ALU, *Arithmetic and Logic Unit*)

1.4 Glossário

A Tabela 3 contém os termos usados neste documento.

Tabela 3. Glossário

Termo	Definição
Active low	O sinal é ativado quando muda o nível lógico para zero
Active high	O sinal é ativado quando muda o nível lógico para um
Asserted	Quando o sinal discreto está no estado de ativação
Negated	Quando o sinal discreto está no estado de desativação
Pin	Conexão física externa

1.5 Notações

A Tabela 4 contém as notações usadas neste documento.

Tabela 4. Notações

Notação	Definição
{ a , b }	Concatenação dos sinais a e b
x'by	Cadeia *y* de *x* bits em formato binário
xd'y	Cadeia *y* de *x* bits em formato decimal
x'hy	Cadeia *y* de *x* bits em formato hexadecimal
Mem[a]	Conteúdo da memória armazenado no endereço *a*
[a : b]	Barramento formado pelo conjunto de todos os bits entre *a* e *b*, incluídos, onde *a* é o bit mais significativo e *b* o menos significativo

2. Introdução

O ChipInventor RVBL-2 é um microcontrolador construído em volta de um núcleo baseado em RISC-V, sendo fabricado usando o PDK Sky130 de 130 nm. Esse guia serve como referência arquitetural para o uso do microcontrolador.

2.1 O Núcleo

O microcontrolador inclui um núcleo RISC-V que implementa o ISA de 32 bits, operando com múltiplos ciclos de *clock* por instrução (ou seja, *multi-cycle*). Essa abordagem permite o compartilhamento de recursos de *hardware* (como interfaces de memória e unidades aritméticas) em diferentes estágios da execução, otimizando a área do chip.

O núcleo é compatível com o padrão RISC-V especificado em [1], implementando o ISA base **RV32I** (para operações inteiras de 32 bits), bem com a extensão padronizada **Zmmul** (derivada da extensão M apenas para instruções de multiplicação). Também implementa uma extensão customizada para calcular o CRC chamada **Xicrc**.

2.2 Descrição dos Pinos do IP

O IP contém apenas 12 pinos, sendo 8 para pinos de propósito geral, 2 para comunicação serial, 1 para sincronização e 1 para (re)inicialização. A Figura 1 contém o diagrama do IP, descrito neste documento, bem como seus pinos:

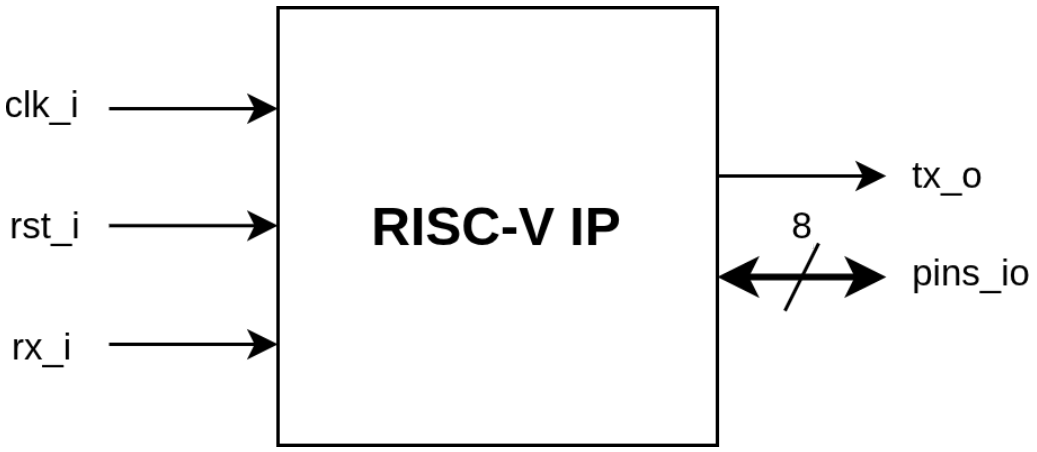


Figura 1. Pinos de interface do IP

A Tabela 5 descreve os sinais de interface do bloco:

Tabela 5. Descrição de Pinos

Nome do Pino	Dir.	Reset	Descrição
rst_i	I	-	Sinal de *reset*. Reinicia a operação do IP.
clk_i	I	-	Sinal de *clock* do núcleo. Guia a operação do IP.

2.3 Estrutura Interna

Além do núcleo baseado em RISC-V, o IP contém outros componentes importantes para o seu funcionamento. A Tabela 6 lista o conjunto dos principais recursos contidos no microcontrolador ChipInventor RVBL-2.

Tabela 6. Conjunto de recursos do IP

Recurso	Descrição
Núcleo (RISC-V)	Núcleo RISC-V implementando o ISA RV32I_Zmmul_Xicrc.
Memória de Instruções (IMEM)	Memória ROM de até 4MB.
Memória de Dados (DMEM)	Memória SRAM de até 8kB.

O núcleo baseado em RISC-V se comunica com os outros módulos (memórias de dados e instruções) via endereços de memória. Portanto, um módulo para decodificação de endereços é utilizado para mediar a comunicação.

A Figura 2 contém o diagrama de alto nível do IP, mostrando seus principais componentes internos e suas interconexões:

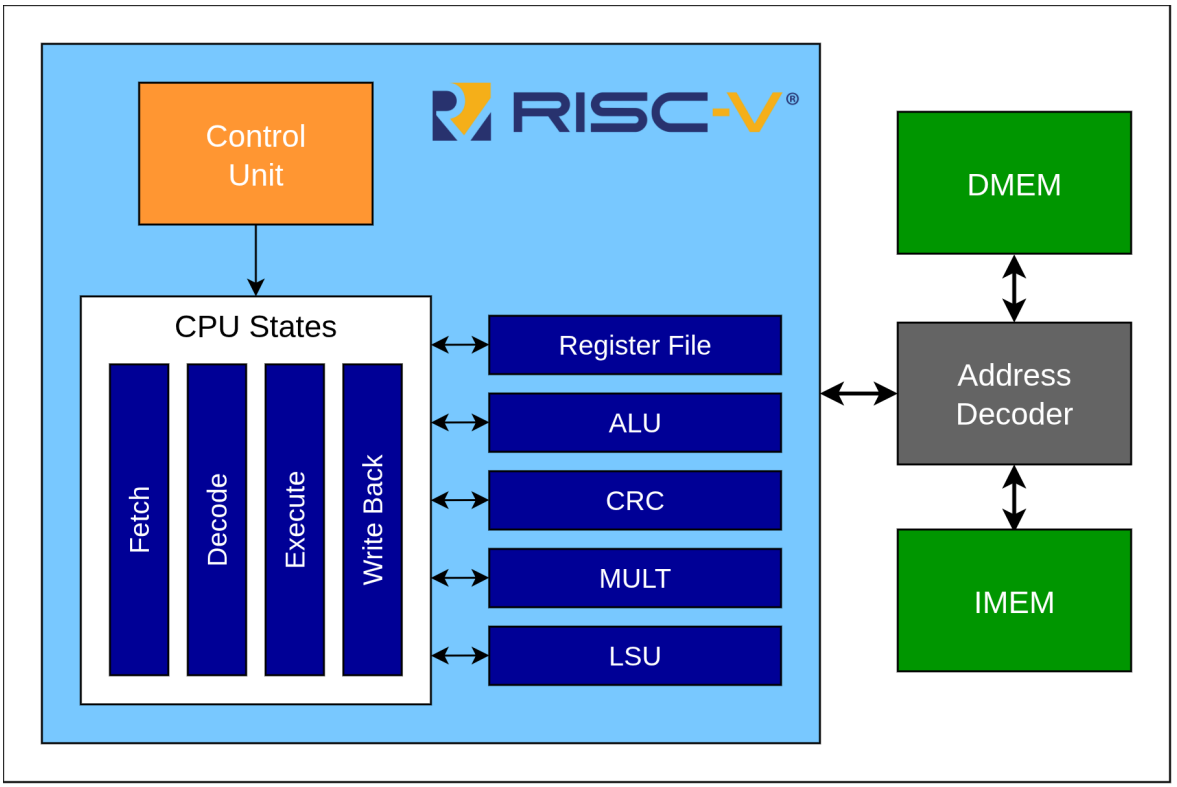


Figura 2. Diagrama de blocos de alto nível do IP

2.4 Conjunto de Instruções

Conforme mencionado anteriormente, o núcleo interno ao microcontrolador implementa o conjunto de instruções base RV32I da arquitetura RISC-V, além de duas extensões adicionais (Zmmul e Xicrc).

De forma a reduzir o tempo necessário para executar operações de multiplicação é necessário adicionar um bloco de *hardware* específico para isso. A extensão do RISC-V que padroniza isso é a M, que especifica instruções de multiplicação e de divisão. Instruções de divisão, porém, aumentam muito a área do chip. De forma a reduzir a área utilizada pelo núcleo foram implementadas as instruções especificadas pela extensão Zmmul, que é um subconjunto da extensão M contendo apenas instruções de multiplicação. Essas instruções podem ser encontradas na Tabela 7, bem como no capítulo **12.3. Zmmul Extension** do manual do conjunto de instruções RISC-V [1]:

Tabela 7. Instruções suportadas pela extensão Zmmul

op	funct3	funct7	Tipo	Instrução	Operação
0110011	000	0000001	R	mul rd, rs1, rs2	$rd = (rs1_{signed} * rs2_{signed})_{31:0}$
0110011	001	0000001	R	mulh rd, rs1, rs2	$rd = (rs1_{signed} * rs2_{signed})_{63:32}$

0110011	010	0000001	R	mulhsu rd, rs1, rs2	rd = $(rs1_{signed} * rs2_{unsigned})_{63:32}$
0110011	011	0000001	R	mulhu rd, rs1, rs2	rd = $(rs1_{unsigned} * rs2_{unsigned})_{63:32}$

O núcleo também é capaz de processar o CRC-16 nativamente por meio de instruções personalizadas, sem a necessidade de um periférico específico, compondo a extensão customizada Xicrc. Isso é possível graças à natureza expansiva do RISC-V. Essas instruções podem ser encontradas na Tabela 8:

Tabela 8. Instruções suportadas pela extensão Xicrc

op	funct3	funct7	Tipo	Instrução	Operação
0110011	000	1000000	R	crcb rd, rs1, rs2	rd = $CRC8(rs1, rs2)_{15:0}$
0110011	001	1000000	R	crch rd, rs1, rs2	rd = $CRC16(rs1, rs2)_{15:0}$
0110011	010	1000000	R	crcw rd, rs1, rs2	rd = $CRC32(rs1, rs2)_{15:0}$

Sendo estas instruções personalizadas o *assembler* do RISC-V não as reconhece, portanto, para serem usadas, diretivas de montagem devem ser usadas para informar os campos **opcode**, **funct3** e **funct7**. Isso possibilita utilizar as instruções, deixando para o assembler o trabalho de traduzi-las para código de máquina. As diretivas usadas são mostradas no código abaixo:

```
.macro crcb rd, rs1, rs2
    .insn r 0x33, 0, 0x40, \rd, \rs1, \rs2
.endm

.macro crch rd, rs1, rs2
    .insn r 0x33, 1, 0x40, \rd, \rs1, \rs2
.endm

.macro crcw rd, rs1, rs2
    .insn r 0x33, 2, 0x40, \rd, \rs1, \rs2
.endm
```

3. Núcleo RISC-V

Essa seção cobre a organização interna do núcleo RISC-V, descrevendo as principais unidades que o compõem. Muitas organizações diferentes possibilitam a implementação da arquitetura RISC-V especificada em [1] de maneira correta, permitindo sua utilização normal.

Sendo assim, essa seção busca trazer a implementação escolhida para este núcleo, que envolve uma máquina de estados utilizando múltiplos ciclos de *clock* para controle da operação do núcleo, bem como implementação das extensões discutidas anteriormente e de outros módulos importantes para a implementação correta das instruções do ISA RV32I.

3.1 *Datapath* do Núcleo

Mantém as principais unidades do núcleo (mostradas na Figura 2), formando o caminho pelo qual os dados trafegam durante a execução das instruções.

3.1.1 ULA

É a unidade responsável por executar todas as operações matemáticas e lógicas do núcleo: adições, subtrações, operações lógicas e deslocamentos de bits. Além de calcular resultados para instruções lógicas/aritméticas, é reutilizada para calcular endereços de memória (em loads/stores) e alvos de desvio (branches/jumps).

É composta primariamente por vários blocos combinacionais que realizam operações sobre as entradas A e B. O resultado de cada bloco é enviado a um multiplexador que seleciona o resultado que deverá ser repassado para a saída com base em um sinal enviado pela unidade de controle. Os valores para este sinal de seleção podem ser vistos na Tabela 9.

Tabela 9. Valores de seleção dos multiplexadores do módulo da ULA

Valor	Operação	Descrição
4'h0	PASS_B	Q = B
4'h1	ADD	Q = A + B
4'h2	SUB	Q = A - B
4'h3	AND	Q = A & B
4'h4	OR	Q = A | B
4'h5	XOR	Q = A ^ B

4'h6	SLL	Q = A << B[4:0]
4'h7	SRL	Q = A >> B[4:0] (*Shift* Lógico)
4'h8	SRA	Q = A >>> B[4:0] (*Shift* Aritmético)
4'h9	SLT	if (A < B) Q = 1, else Q = 0
4'hA	SLTU	if ($A_{unsigned}$ < $B_{unsigned}$) Q = 1, else Q = 0

3.1.2 Multiplicador

É a unidade responsável por executar as operações de multiplicação do núcleo.

É composta por multiplexadores que selecionam quais entradas devem ser mandadas para o bloco que irá realizar a multiplicação (de 64 bits), além de outro multiplexador que seleciona qual parte do resultado final deve ser mandada para a saída (os 32 bits inferiores ou superiores).

Os valores dos sinais de seleção destes multiplexadores podem ser vistos na Tabela 10.

Tabela 10. Valores de seleção dos multiplexadores do módulo de MULT

Valor	Operação	Descrição
4'h0	MUL	rd = $(rs1_{signed} * rs2_{signed})_{31:0}$
4'h1	MULH	rd = $(rs1_{signed} * rs2_{signed})_{63:32}$
4'h2	MULHSU	rd = $(rs1_{signed} * rs2_{unsigned})_{63:32}$
4'h3	MULHU	rd = $(rs1_{unsigned} * rs2_{unsigned})_{63:32}$

De acordo com a instrução a ser executada deve ser selecionados os sinais A e B com ou sem sinal para serem multiplicados. Além disso, deve ser selecionado os 32 bits inferiores ou superiores do resultado.

O multiplicador, então, utiliza um sinal vindo da unidade de controle para realizar a seleção dos valores corretos nos três multiplexadores (A, B e resultado).

3.1.3 CRC

Unidade responsável por realizar o processamento do CRC de 16 bits internamente ao núcleo, sem latência. É composta por três blocos, onde cada um realiza o CRC com uma parte das entradas (8, 16 ou 32 bits) e por um multiplexador, que seleciona o valor final dos blocos e o repassa para a saída, com base em um sinal de seleção vindo da unidade de controle.

Os valores dos sinais de seleção destes multiplexadores podem ser vistos na Tabela 11.

Tabela 11. Valores do multiplexador interno do módulo de CRC

Valor	Operação	Descrição
4'h0	CRCB	rd = CRC8(rs1, rs2)$_{15:0}$
4'h1	CRCH	rd = CRC16(rs1, rs2)$_{15:0}$
4'h2	CRCW	rd = CRC32(rs1, rs2)$_{15:0}$

De acordo com a instrução a ser executada devem ser selecionadas partes dos sinais A e B para ser computado o CRC de 16 bits.

3.1.4 Registradores do RISC-V

O núcleo RISC-V possui 34 registradores: 32 de propósito geral (no banco de registradores) à disposição do usuário, o PC (contador de programa) e o IR (registrador de instrução).

O **banco de registradores** mantém os dados temporários da CPU. Contém 32 registradores de propósito geral de 32 bits (x0 a x31). O registrador x0 é hardwired (fixo) em zero, não podendo ser modificado. Permite leitura assíncrona simultânea dos operandos rs1 e rs2, e escrita síncrona do destino rd no estágio de *Write Back*. Os sinais de seleção dos registradores são extraídos diretamente da instrução, e o sinal de *write enable* é enviado pela unidade de controle

O **contador de programa** (PC) rastreia o fluxo de execução, mantendo o endereço de 32 bits da instrução atual. É atualizado sequencialmente (PC + 4) a cada ciclo de *clock* ou carregado com novos endereços derivados de instruções de controle de fluxo (*branch* e *jump*). O núcleo RISC-V usa o valor de PC para buscar a próxima instrução a ser executada.

O **registrador de instrução** (IR) mantém os 32 bits referentes à instrução atual a ser executada, buscada anteriormente no ciclo de *Fetch*. Seu conteúdo é lido pelos outros blocos e decodificado para permitir a execução correta da instrução.

3.1.5 Extensão de Imediatos

Módulo responsável por extrair e formatar os valores imediatos embutidos nas instruções RISC-V, sendo completamente combinacional. Sendo assim, decodifica as instruções de tipo I, S, B, U e J com base em seu *opcode* e extrai seu imediato.

Portanto, pega os bits espalhados pela instrução (com base na formatação do tipo específico da instrução) e realiza a extensão de sinal para gerar um operando de 32 bits válido para a ULA.

3.1.6 Comparador de Branch

Unidade responsável por executar as instruções de salto condicional (ou *branch*). Esse módulo compara os registradores rs1 e rs2 vindos do banco de registradores com base na instrução de *branch* (selecionadas pelo campo **funct3** da instrução). O módulo, ainda, precisa se preocupar se a comparação leva em consideração o sinal ou não.

Sendo um módulo combinacional, este precisa obter os sinais de entrada, junto à seleção, realizar a comparação desejada e informar se o salto será tomado ou não.

3.2 Unidade de Controle

Componente responsável por controlar a operação do núcleo RISC-V, sendo seu componente mais crítico. Para tanto, implementa internamente uma Máquina de Estados Finita (FSM) que controla o datapath, sinalizando quais unidades devem ser usadas e como devem ser usadas.

Assim, a FSM interna divide a execução nas seguintes fases, cada uma correspondendo a um ciclo de *clock*:

- **Busca de Instrução (*Fetch*):** Lê a instrução da Memória ROM usando o valor atual do PC e salva no registrador de instrução (IR).
- **Decodificação da Instrução (*Decode*):** Lê os operandos do Banco de Registradores e decodifica a instrução para determinar os próximos estados.
- **Execute (Execução):** A ULA entra em ação para realizar o cálculo principal da instrução, calcular o endereço efetivo de memória ou avaliar a condição de um desvio (Branch).
- **Write Back (Escrita de Retorno):** Salva o resultado final (vindo da ULA ou da Memória RAM via LSU) no registrador de destino no Banco de Registradores.

A unidade de controle, então, recebe o ***opcode*** da instrução e emite sinais que irão controlar o datapath, permitindo sua execução.

3.3 LSU (Load/Store Unit)

Esse módulo fornece uma interface direta entre o núcleo RISC-V e as memórias. Estes dispositivos podem ser acessados pelo núcleo RISC-V por meio de endereços, conforme comentado na seção **4. Mapeamento de Memória**. Esses endereços mapeiam posições internas aos dispositivos que podem ser lidas ou escritas pelo núcleo.

Sendo assim, a LSU possui duas funções principais: gerenciar as operações de **leitura** (ou *load*) e **escrita** (ou *store*) de dados.

3.3.1 Instruções de *Load*

A operação de leitura consiste no núcleo RISC-V obter o valor de uma posição utilizando seu endereço de memória e colocá-lo em um registrador interno. O núcleo RISC-V, sendo uma arquitetura de 32 bits, trabalha primariamente com palavras de 4 bytes, onde os endereços referenciam palavras de 4 bytes, já que este é o tamanho dos registradores do núcleo. O núcleo, porém, pode acessar "pedaços" desta palavra (de 1 ou 2 bytes).

Abaixo estão listadas as instruções de *load* do RISC-V:

- **lw:** Lê uma palavra de 4 bytes.
- **lh:** Lê uma palavra de 2 bytes (com sinal)
- **lb:** Lê uma palavra de 1 byte (com sinal).
- **lhu:** Lê uma palavra de 2 bytes (sem sinal).
- **lbu:** Lê uma palavra de 1 byte (sem sinal).

Instruções que acessam menos de 4 bytes precisam escolher como irão preencher o restante do registrador. As instruções **lh** e **lb** preenchem o restante dos bits do registrador com o valor do bit de sinal (MSB) do valor lido (extensão de sinal). Dessa forma, ao ler um valor de 1 byte negativo (com o bit 7 igual a 1) os outros 3 bytes do registrador serão preenchidos com valor 1, mantendo o valor negativo.

Já as instruções **lhu** e **lbu** preenchem o restante dos bits do registrador com zero (extensão de zeros). Dessa forma, ao ler um valor de 1 byte que não possua sinal (o bit 7 não deva representar um sinal) os outros 3 bytes do registrador serão preenchidos com valor 0, mantendo o valor integral.

A Tabela 12 traz um exemplo da memória de dados para demonstrar a funcionalidade das diferentes instruções de *load*. A primeira coluna possui o endereço cada byte da DMEM (mostrado na terceira coluna), enquanto a segunda coluna possui o endereço de cada palavra (definido pelo módulo).

Tabela 12. Exemplo de posições da DMEM

Endereço Absoluto	Endereço Real da DMEM	Conteúdo
0x10010000	0x0000	0xF1
0x10010001		0xF2
0x10010002		0xF3
0x10010003		0xF4
0x10010004	0x0004	
...		

Abaixo está o resultado após carregar o valor no endereço 0x10010000 usando **lbu** (carrega 1 byte de 0x10010000, preenchendo o restante do registrador com zeros):

00	00	00	F1
3	2	1	0

Abaixo está o resultado após carregar o valor no endereço 0x10010000 usando **lb** (carrega 1 byte de 0x10010000, estendendo o sinal de 0xF1 para o restante do registrador):

FF	FF	FF	F1
3	2	1	0

Abaixo está o resultado após carregar o valor no endereço 0x10010000 usando **lhu** (carrega 2 bytes a partir de 0x10010000, preenchendo o restante do registrador com zeros):

00	00	F2	F1
3	2	1	0

Abaixo está o resultado após carregar o valor no endereço 0x10010000 usando **lh** (carrega 2 bytes a partir de 0x10010000, estendendo o sinal de 0xF2 para o restante do registrador):

FF	FF	F2	F1
3	2	1	0

Abaixo está o resultado após carregar o valor no endereço 0x10010000 usando **lw** (carrega 4 bytes a partir de 0x10010000):

F4	F3	F2	F1
3	2	1	0

Sendo assim, a LSU deve controlar a operação de *load*, estabelecendo qual parte da palavra lida da memória será realmente repassada ao registrador.

Por exemplo, ao ler o endereço 0x10010002, apenas o byte 0xF3 da palavra lida (no caso 0xF4**F3**F2F1) será colocado no registrador (que deverá então ter o valor 0x000000**F3**). A LSU, então, deve reposicionar o byte corretamente. O 2 bits inferiores do endereço são usados para informar qual byte está sendo lido.

Vale lembrar que **lw** é alinhado em 4 bytes, **lh** e **lhu** em 2 bytes e **lb** e **lbu** em 1 byte.

3.3.2 Instruções de *Store*

A operação de escrita consiste no núcleo RISC-V escrever o valor de um registrador interno em uma posição utilizando seu endereço de memória. Assim como no *load* o núcleo pode escrever apenas "partes" da palavra de memória.

Abaixo estão listadas as instruções de *store* do RISC-V:

- **sw:** Escreve uma palavra de 4 bytes.
- **sh:** Escreve uma palavra de 2 bytes.
- **sb:** Escreve uma palavra de 1 byte.

Essas instruções, então, fornecem ao RISC-V capacidade de escrever uma palavra inteira na memória (com **sw**) ou partes menores (com **sh** e **sb**). Diferentemente de instruções de *load* não há necessidade de extensão do valor, já que, por mais que a memória seja dividida em palavras de 4 bytes, cada byte desta palavra é endereçável.

Sendo assim, a LSU deve controlar a operação de *store*, estabelecendo qual parte da palavra de memória deve ser modificada. A LSU emite, então, um sinal chamado ***byte write***, de 4 bits, onde cada um define qual dos quatro bytes da palavra será escrito. Além disso, deve posicionar o dado a ser escrito na posição desejada da palavra. Vale lembrar que **sw** é alinhado em 4 bytes, **sh** em 2 bytes e **sb** em 1 byte.

Por exemplo, para escrever o dado 0x12 no endereço 0x10010002 (da Tabela 13) a LSU deve informar que o terceiro byte da palavra na posição 0x0000 será escrito (com *byte write* igual a 0100_2), posicionando o byte de maneira correta (ficando 0x00120000). Assim, a DMEM irá escrever apenas apenas o terceiro byte!

3.3.3 Interface da LSU

Trata o acesso de palavras completas (32 bits), meias palavras (16 bits) e bytes (8 bits), aplicando a extensão de sinal ou de zeros adequada nas operações de Load.

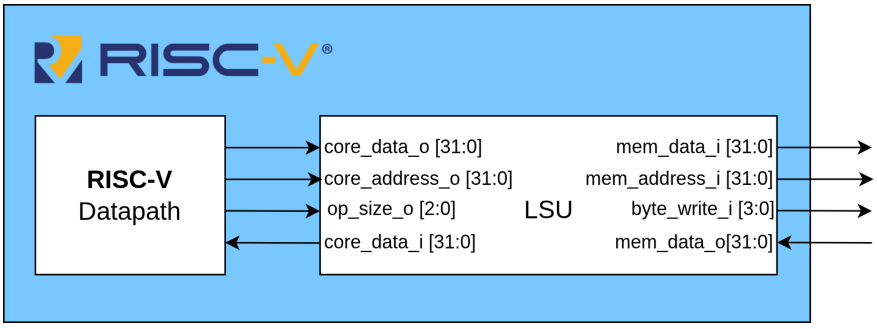


Figura 3. Diagrama de blocos da relação entre o núcleo e a LSU

4. Mapeamento de Memória

Essa seção cobre o mapeamento de memória do IP, bem como a operação dos equipamentos de memória e do decodificador de endereços, que implementa o mapeamento dos endereços de memória dos dispositivos.

4.1 Mapa de Memória

Este dispositivo contém os dispositivos mapeados em endereços de memória localizados em um espaço de endereçamento de 4 GB. A Tabela 13 descreve as localizações dos dispositivos dentro desse espaço de memória.

Tabela 13. Mapa de memória do sistema

Endereço Base	Descrição
0x00400000	Memória de Instruções (IMEM) (de 4 MB)
0x10010000	Memória de Dados (DMEM) (de 8 kB)

4.2 Memória de Instruções (IMEM)

Essa memória contém o programa (ou *firmware*) a ser lido e executado pelo núcleo RISC-V.

A IMEM possui uma implementação simples, sendo apenas uma **ROM** contendo as instruções a serem executadas. Ao receber o endereço de memória a ser acessado e um sinal de leitura (sinal *output enable*), a instrução de 32 bits que estava naquela posição é recebida pelo núcleo para ser executada.

Tabela 14. Exemplo de *Firmware* na IMEM

Endereço	Conteúdo
0x00400000	0x00842283
0x00400004	0xf0000437
...	...
0x004000A0	0x00032283
...	...
0x00400100	0x0000000A

...	...

Além de manter as instruções, a IMEM pode manter **valores constantes** que podem ser lidos pela CPU durante a execução de um programa. Por exemplo, a instrução

0x004000A0 : 0x00032283 : **lw** t0, 0(t1)

está localizada na IMEM, no endereço 0x004000A0. Portanto, quando a CPU enviar esse endereço no ciclo de busca, receberá o valor 0x00032283 (hexadecimal equivalente à instrução guardado na memória), que corresponde à instrução de *load*. Ao executar, usará **t1** como ponteiro de memória, e, supondo que **t1** tenha valor 0x00400100 a IMEM será acessada novamente, mas não para ler uma instrução, mas sim um valor constante, que de acordo com a Tabela 14 é o valor 10.

A IMEM, ainda, contém endereços alinhados em 4 bytes, isto é, com os dois bits inferiores sempre zero (por exemplo 0x00400000, 0x00400004, 0x00400008, etc).

4.3 Memória de Dados (DMEM)

Essa memória contém os dados utilizados no decorrer da execução do programa pelo núcleo RISC-V, as **variáveis**. É na memória DMEM que ficam as variáveis:

- **Globais**: Em uma parte da memória chamada **.data**.
- **Não-inicializadas**: Em uma parte da memória chamada **.bss**.
- **Locais** (ou temporárias): Em uma parte da memória chamada ***stack*** (ou pilha).
- **Dinâmicas**: Em uma parte da memória chamada ***heap***.

A DMEM é implementada usando células de memória SRAM com leitura e escrita **síncronas**. Assim, ao realizar uma escrita (via instruções **sw**, **sh** e **sb**) ou uma leitura (via instruções **lw**, **lh**, **lb**, **lhu** e **lbu**) a operação só será concluída após um ciclo de *clock*.

A DMEM, internamente, endereça palavras de 4 bytes ao invés de 1 byte, isto é, contém endereços alinhados em 32 bits, ignorando os dois bits inferiores do endereço (por exemplo 0x10010000, 0x10010004, 0x10010008, etc). Mesmo assim, a memória deve permitir a escrita independente de cada byte destas palavras de 4 bytes (via instruções **sh** e **sb**), conforme manda a seção **3.3. LSU (Load/Store Unit)** da especificação do RISC-V [1].

Para possibilitar isso, a memória mantém *n* palavras de 4 bytes, onde *n* é o tamanho total da memória em bytes dividido por 4. Sendo assim, uma memória de 256 bytes contém 64 palavras de 4 bytes, por exemplo. O núcleo, então, ao executar **sh** (*store half*, escreve 2 bytes) ou **sb** (*store byte*, escreve 1 byte) solicita (via **bw_o** da LSU) que partes de uma palavra possam ser escritas. A memória, então, deve usar esse sinal para selecionar quais bytes da palavra memória serão escritos.

4.4 Decodificador de Endereços

Este componente atua como um intermediário entre o núcleo RISC-V e os dispositivos de memória, todos acessados por meio de endereços de memória (estratégia conhecida por MMIO) e instruções de *load* e *store*, isolando o espaço de endereçamento de vários dispositivos.

Seu funcionamento se baseia em receber um endereço de memória do núcleo RISC-V e verificar qual dispositivo está mapeado para aquele endereço, selecionando-o para a operação de *load* ou *store*, roteando os sinais necessários de acordo com o dispositivo (descritos a seguir).

A Figura 4 contém o diagrama de blocos do decodificador de endereços, mostrando seus principais componentes internos e suas interconexões:

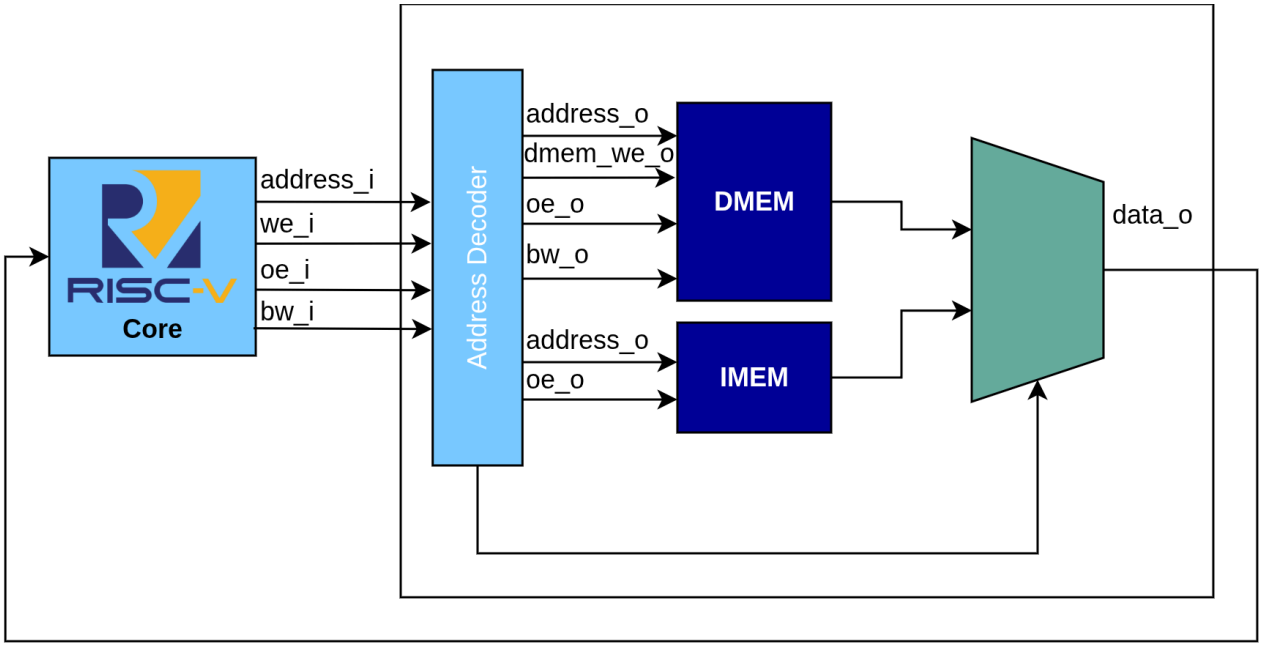


Figura 4. Diagrama de blocos do decodificador de endereços

O núcleo emite quatro sinais de controle diferente (além dos barramentos de entrada e saída de dados), que devem ser roteados para os dispositivos pelo decodificador:

- Emite o sinal de **endereço** (provindo da porta **address_o** do núcleo) para todos os dispositivos, ignorando os dois bits inferiores (alinhamento em 4 bytes).
- Emite o sinal de **escrita** (*Write Enable*, provindo da porta **we_o** do núcleo) para todos os dispositivos, COM EXCEÇÃO da memória IMEM, que não pode ser escrita.
- Emite o sinal de **leitura** (*Output Enable*, provindo da porta **oe_o** do núcleo) para todos os dispositivos.

- Emite o sinal de **seleção dos bytes a serem escritos** (*Byte Write*, provindo da porta **bw_o** do núcleo) APENAS para a memória DMEM, de forma a selecionar os bytes que serão escritos.

Dito isso, o módulo deve rotear os seguintes sinais do núcleo para todos os dispositivos: **address_o** (com exceção dos 2 bits inferiores), **bw_o** (apenas para a DMEM), **we_o** (com exceção da IMEM) e **oe_o**, conforme mostra a Figura 4.

Além disso, o módulo deve analisar o endereço enviado pelo núcleo, com base nos endereços de cada dispositivo, conforme consta na Tabela 13, e selecionar qual dos dados de saída dos módulos será enviado para o núcleo, por meio de um sinal de seleção e de um multiplexador.


ChampionCHIP
eXperience